Moulinette Lab

GUIDE UTILISATEUR · VERSION 0.32.5-DEV

Installer, corriger et comprendre sa progression

Manuel du prototype local pour les piscines C, C++ et Python. Deux versions de ce guide existent : Blanc et Océan ; leur contenu est identique.

Projet non officiel, sans affiliation à 42. Les tests sont partiels et un succès ne garantit ni conformité complète ni résultat officiel. Les sujets et versions de référence C++ et Python ne sont pas confirmés.

Sécurité : utilisez uniquement votre propre code ou des sources de confiance que vous avez examinées. Aucun mode ne protège totalement contre du code hostile ; scripts et Makefiles peuvent exécuter des commandes.

01 · Présentation

Moulinette Lab évalue un rendu local de confiance, produit un rapport et enregistre les tentatives dans une base locale. Un tableau de bord permet de suivre l'état des exercices, les détails des corrections et leur comparaison. Un dépôt GitHub public de confiance peut aussi être récupéré avec votre accord ; cette opération utilise le réseau.

Le catalogue contient 109 exercices C, 45 C++ et 36 Python, soit 190 identifiants. Les contrôles ne sont ni officiels ni exhaustifs et ne prouvent pas la conformité au sujet. Les moteurs C++ et Python utilisent des contrôles distincts : ils ne sont pas la Norminette C. Le Lab ne fournit pas les tests officiels et ne fabrique pas automatiquement des tests à partir d'un sujet PDF.

Parcours de ce manuel

- 02 · Préparer et installer le Lab
- 03 · Premières corrections
- 04 · Rapport et sécurité
- 05 · GitHub de confiance
- 06 · Tableau de bord, piscines et historique
- 07 · Référence des commandes et options
- 08 · Dépannage et limites

02 · Installation et prérequis

Ces exemples visent Ubuntu ; adaptez les chemins à votre installation. Pour C, prévoyez Python 3.10 ou plus, un compilateur C, make, ar et nm. Pour C++, il faut aussi un compilateur C++. Bubblewrap est utilisé par défaut s'il est disponible ; Git est nécessaire pour GitHub. La Norminette 3.3.59 doit être accessible pour les contrôles C concernés.

Si les outils manquent, leur installation utilise le réseau et demande les droits administrateur :

```
sudo apt update
sudo apt install python3 python3-venv build-essential \
  binutils make bubblewrap git xz-utils
command -v bwrap
command -v norminette
norminette --version
```

Si la Norminette manque, un environnement virtuel dédié est possible :

```
python3 -m venv ~/.venvs/moulinette-norm
source ~/.venvs/moulinette-norm/bin/activate
python -m pip install 'norminette==3.3.59'
```

Réactivez cet environnement dans un nouveau terminal si la commande norminette n'est plus accessible. Ne réinstallez rien si vos outils fonctionnent déjà.

02 · Installation, suite

Extraire sans écraser une ancienne version

Téléchargez `moulinette-lab-v0.32.5-dev.tar.xz`. L'exemple suppose que le fichier se trouve directement dans Téléchargements ; s'il est dans un autre dossier, adaptez le chemin. L'extraction utilise un nouveau dossier dans votre espace de travail :

```
mkdir -p "$HOME/Téléchargements/workspace_moulinette_arena"
ESSAI=$(mktemp -d "$HOME/Téléchargements/workspace_moulinette_arena/lab-XXXXXX")
tar -xf "$HOME/Téléchargements/moulinette-lab-v0.32.5-dev.tar.xz" -C "$ESSAI"
cd "$ESSAI/moulinette-lab-v0.32.5-dev"
```

Arrêtez-vous si tar indique une erreur. Vérifiez que le nom du fichier et son emplacement sont corrects avant de continuer. La variable ESSAI est propre au terminal courant ; le dossier créé reste sur le disque. Notez son chemin si vous souhaitez le retrouver.

Installer la commande

Depuis la racine extraite, activez l'environnement contenant la Norminette, puis installez le Lab dans cet environnement. La première ligne ci-dessous vaut si vous avez créé l'environnement dédié indiqué plus haut ; sinon, adaptez-la à votre installation. L'installation peut nécessiter des outils de construction ou une connexion.

```
source ~/.venvs/moulinette-norm/bin/activate
python -m pip install .
moulinette --version
moulinette --help
```

La version attendue est `0.32.5.dev0`. `command -v moulinette` indique la commande exécutée. En cas d'ancienne version, vérifiez l'environnement activé. `deactivate` le quitte. `moulinette/` contient le programme et ses ressources ; `exemples/` contient des rendus de démonstration ; `docs/` contient les guides et leurs sources.

03 · Premières corrections

Exemple local fourni

Depuis la racine du projet extrait, utilisez l'exemple de confiance :

```
moulinette check ./exemples/c01/ex00 \
  -e c01/ex00 -s
```

Un raccourci équivalent est `moulinette ./exemples/c01/ex00 -e c01/ex00 -s`. `-s` sélectionne Bubblewrap expérimental ; `-e` fixe l'identifiant. Un résultat réussi signifie uniquement que les contrôles exécutés ont réussi.

Votre travail

```
moulinette /chemin/vers/mon/rendu -e c00/ex00 -s
```

Le chemin peut désigner un dossier ou un fichier local. La détection automatique ne fonctionne pas pour tous les noms ou chemins : en cas de doute indiquez `-e`. Respectez les fichiers exigés par le sujet. Pour un exercice de fonction, gardez vos essais personnels séparés du rendu. N'évaluez que du code de confiance.

La commande `moulinette` seule ouvre le tableau de bord. Pour afficher seulement son chemin et l'ouvrir vous-même :

```
moulinette dashboard --no-open
```

Après une correction, régénérez le tableau de bord statique et rechargez la page. Le mode live possède un fonctionnement distinct décrit plus loin.

04 · Comprendre les résultats

- **Réussi** : les contrôles exécutés ont réussi ; aucune conformité complète n'est garantie.
- **Échec / à revoir** : un contrôle a échoué ; d'autres peuvent ne pas avoir été exécutés.
- **Incomplet** : contrôle indisponible, résultat ambigu ou outil manquant ; ce n'est pas une réussite.
- Les informations du rapport ne sont ni des réussites ni des échecs.

Pour `check`, les codes de sortie sont 0 pour réussi, 1 pour échec et 2 pour incomplet ou erreur d'utilisation. Les diagnostics pédagogiques, par exemple variable inutilisée, retour manquant ou prototype attendu, dépendent du rapport et restent indicatifs.

La Norminette C 3.3.59 est adaptée au profil autodidacte : seul `INVALID_HEADER` est ignoré. Un outil absent ou incompatible ne valide pas le style. La compilation, le style et les tests fonctionnels contrôlent des aspects différents. C05 peut afficher un indice textuel non bloquant sur la récursivité : il ne prouve pas qu'une fonction est ou n'est pas récursive.

Modes d'exécution

Sans option de mode, Bubblewrap est utilisé s'il est installé ; sinon la correction est refusée. `-s` / `--experimental-sandbox` le demande explicitement. `--cgroup` / `--experimental-cgroup-sandbox` ajoute des limites expérimentales avec systemd et ne prend en charge que C. `-l` / `--local` / `--unsafe-local` exécute sans isolation avec les droits de votre session. Ces modes sont exclusifs.

Aucun de ces modes ne rend sûr un code hostile. Ne modifiez pas les protections système pour forcer un essai. Les limites de capture, de temps, Bubblewrap et cgroup ne constituent pas une garantie de sécurité.

05 · GitHub de confiance

Seuls les dépôts GitHub publics HTTPS sont prévus. Vérifiez le contenu, y compris scripts et Makefiles, avant de le choisir. `--trust-github` exprime votre accord pour récupérer ce dépôt ; l'accès réseau concerne l'hôte et le parcours reste expérimental.

```
moulinette check https://github.com/UTILISATEUR/DEPOT \
  -e c05/ex01 --subdir c05/ex01 --trust-github -s
```

Remplacez l'URL, l'exercice et le sous-dossier par les vôtres. Sans `--subdir`, la racine est évaluée ; le sous-dossier est relatif, sans `..` ni lien symbolique dans ses composants. `--ref` accepte une branche, un tag ou un SHA complet de 40 caractères ; sans option, le HEAD par défaut est utilisé. Les SHA abrégés ne sont pas acceptés.

Le Lab travaille sur une copie temporaire ; cela ne prouve pas la sécurité du code. Dépôts privés, SSH, sous-modules, Git LFS et redirections ne sont pas pris en charge. Une récupération indisponible ne signifie pas que la correction a réussi.

06 · Tableau de bord et piscines

Le parcours présenté est C, puis C++, puis Python. Une carte évolutive montre la piscine consultée. Les piscines suivantes se débloquent dans cet ordre ; le déblocage est permanent, même après une régression. Le verrouillage concerne la page : la CLI reste utilisable et les réussites anticipées sont conservées.

Chaque exercice utilise son dernier résultat : réussi, à revoir, incomplet ou non corrigé. Les 50 dernières corrections et leurs détails sont affichés. Une comparaison avec la tentative précédente peut être proposée parmi ces corrections affichées ; les contrôles non comparables ne sont pas réputés réussis.

L'atome centré derrière la carte comporte un noyau et une orbite avec un électron pour chaque exercice réussi ou à revoir dans le périmètre. Bleu signifie réussi, rouge signifie à revoir ; incomplets et non corrigés n'ajoutent rien. Le périmètre suit la piscine consultée ou le module choisi ; la recherche textuelle ne le change pas. Des électrons et orbites peuvent se superposer ou être cachés derrière la carte. Le mouvement réduit est respecté.

Thèmes, humour et documents

Le thème manuel et le choix d'humour sont mémorisés si le stockage local du navigateur est disponible. Le mode aléatoire choisit un thème à l'ouverture. Lorsque l'humour est activé et l'onglet visible, la phrase change toutes les 30 secondes ; le bouton permet aussi de la changer manuellement. Le stockage peut être refusé par le navigateur.

Les deux guides distincts sont disponibles en Blanc et Océan par quatre boutons de téléchargement direct. Ce sont des documents généraux, pas des bilans personnalisés. Ils peuvent être ouverts hors ligne dans un lecteur PDF.

06 · Données et direct

Le score est le nombre de contrôles réussis divisé par les contrôles réussis et échoués effectivement exécutés ; les informations sont exclues. Si le résultat est incomplet ou un contrôle indisponible, aucun score n'est affiché. Ce n'est pas une note officielle.

Par défaut, l'historique se trouve dans `~/.local/share/moulinette-lab/history.sqlite3`. Si `XDG_DATA_HOME` est un chemin absolu, les données sont sous ce dossier. Les diagnostics peuvent contenir du code et des chemins personnels : vérifiez-les avant de partager la base ou la page. `--no-history` évite l'enregistrement de la correction concernée.

Pour un essai séparé, définir `XDG_DATA_HOME` vers un nouveau dossier pour toutes les commandes de cet essai. Ne prenez pas `dashboard --db` comme moyen d'isoler votre historique : une limite connue peut utiliser ou modifier la base par défaut pour les déblocages ; le mode live peut aussi ignorer la base personnalisée.

Le mode `moulinette dashboard --live` lance un serveur local sur 127.0.0.1, avec jeton de session, et propose une correction depuis la page ainsi qu'un suivi. Une page statique peut nécessiter une régénération et un rechargement. Protégez le lien local ; le direct ne garantit pas une isolation complète.

07 · Référence des commandes

Aide, version et catalogue

```
moulinette --help
moulinette --version
moulinette check --help
moulinette dashboard --help
moulinette list --piscine c
moulinette list --piscine cpp
moulinette list --piscine py
moulinette list --piscine all
```

`moulinette` seul, `moulinette dashboard` et `moulinette ui` ouvrent le tableau de bord. `moulinette CHEMIN` équivaut à `moulinette check CHEMIN`. Le chemin de correction est obligatoire, y compris avec `--all`.

Sélection et exécution

- `-e`, `--exercise` : identifiant explicite, par exemple c01/ex00 ou cpp00/ex00.
- `-m`, `--module` : évaluer les exercices présents d'un module dans un dossier approprié.
- `-a`, `--all` : parcourir les exercices présents d'un dossier racine. Ces trois sélections sont mutuellement exclusives.
- `-j`, `--jobs N` : 1 à 8 travailleurs pour module ou all, 1 par défaut.
- `-w`, `--watch` : surveiller les fichiers et relancer ; `--watch-interval` fixe l'intervalle en secondes. Arrêt avec Ctrl+C.
- `-s`, `--sandbox`, `--experimental-sandbox` : Bubblewrap expérimental.
- `--cgroup`, `--experimental-cgroup-sandbox` : Bubblewrap et cgroup expérimentaux, C uniquement.
- `-l`, `--local`, `--unsafe-local` : mode local non isolé ; code de confiance uniquement.
- `--sanitize` : passe optionnelle de diagnostic mémoire et UB. Pour les moteurs C++ et Python, sa prise en charge n'est pas établie ; une absence d'erreur ne prouve pas l'absence de défauts.
- `--json` : rapport JSON ; `--no-history` : ne pas enregistrer la tentative.
- `--trust-github`, `--ref` et `--subdir` : options réservées à GitHub, décrites dans la section 05.

07 · Commandes, suite

Exemples

```
moulinette /chemin/vers/c01 -m c01 -s
moulinette /chemin/vers/rendus -a -j 1 -s
moulinette ./exemples/c01/ex00 -e c01/ex00 -s \
  --json --no-history
moulinette /chemin/vers/mon/rendu -e c00/ex00 -w -s
```

Adaptez les chemins ; tous les exemples supposent des fichiers de confiance. La surveillance peut relancer le programme plusieurs fois.

Options du tableau de bord

```
moulinette dashboard --no-open
moulinette dashboard --output /chemin/dashboard.html --no-open
moulinette dashboard --live
moulinette dashboard --live --port 8765
```

`--no-open` ne lance pas le navigateur ; `--open` demande son ouverture ; `--output` choisit un fichier HTML ; `--live` lance le mode direct ; `--port` choisit son port, 0 par défaut pour un port automatique. `--db` existe mais son isolation est limitée : lire la section 06 avant de l'utiliser.

Diagnostics optionnels

```
moulinette sandbox doctor
moulinette sandbox selftest
moulinette sandbox cgroup-selftest
```

Chaque commande accepte `--json`. Doctor examine des prérequis ; selftest exécute des témoins fixes et cgroup-selftest atteint volontairement une petite limite de tâches : ce dernier n'est pas un test sans charge. Ces sondes sont facultatives et ne prouvent pas la sécurité. Ne lancez pas de témoin OOM ou de saturation sur votre session de travail, notamment celui de l'ancienne v0.20.

08 · Dépannage et limites

Archive ou commande introuvable

Si tar indique « Aucun fichier ou dossier », vérifiez le nom et le dossier réel de l'archive ; l'extraction n'a pas réussi. N'exécutez pas les étapes suivantes avant correction. Si `moulinette` est introuvable ou si une ancienne version apparaît, vérifiez l'environnement virtuel activé et `command -v moulinette`. Vérifiez aussi que le projet a bien été extrait.

Norminette, Bubblewrap ou résultat incomplet

Vérifiez `command -v norminette`, `norminette --version` et `command -v bwrap`. Une Norminette absente ou incompatible peut empêcher un verdict complet. Une erreur Bubblewrap n'autorise pas implicitement le mode local ; ne choisissez `-l` qu'en comprenant l'absence d'isolation et pour du code de confiance.

Tableau de bord

Avec `--no-open`, ouvrez le fichier affiché dans votre navigateur. Vérifiez le même navigateur et le même chemin si une préférence ne persiste pas. Une page statique doit être régénérée après correction. Un message du navigateur ne prouve pas à lui seul que la page s'est ouverte ou non.

Limites des tests

Les tests fonctionnels, de style, de compilation et d'analyse ne couvrent pas tous les cas, ni toute la sécurité ou la mémoire. La conformité C++/Python aux sujets et l'identité de leurs versions ne sont pas établies ; les contrôles de ces moteurs sont particulièrement partiels. Consultez les sujets que vous possédez et le rapport détaillé. Les résultats de versions antérieures ou d'autres environnements ne prouvent pas ceux de votre installation actuelle.

Pour les détails : `README.md`, les documents dans `docs/`, et le guide de style C séparé. Les polices intégrées sont accompagnées de leurs licences dans le projet.